

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА»
(СПбГУТ)

АРХАНГЕЛЬСКИЙ КОЛЛЕДЖ ТЕЛЕКОММУНИКАЦИЙ
ИМ. Б.Л. РОЗИНГА (ФИЛИАЛ) СПбГУТ
(АКТ (ф) СПбГУТ)

Допущен к защите

Зав. отделением

 Ю.В. Солодкая
(Подпись) (И.О. Фамилия)

« 6 » июня 2026 г.

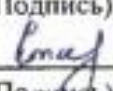
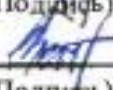
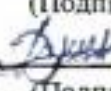
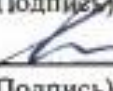
ДИПЛОМНЫЙ ПРОЕКТ НА ТЕМУ

РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ
ДЛЯ ИЗУЧЕНИЯ ПРАВИЛ РУССКОГО ЯЗЫКА

«ГРАММАРИУМ»

Л109. 26ДП00. 002 ПЗ

(Обозначение документа)

Рецензент		06.06.2026	И.Л. Самоукова
	(Подпись)	(Дата)	(И.О. Фамилия)
Руководитель		06.06.2026	В.В. Старостина
	(Подпись)	(Дата)	(И.О. Фамилия)
Дипломник		06.06.2026	А.М. Вепрёв
	(Подпись)	(Дата)	(И.О. Фамилия)
Консультант по оформлению		06.06.2026	Д.П. Ушаков
	(Подпись)	(Дата)	(И.О. Фамилия)
Консультант по экономической части		06.06.2026	С.Н. Короткова
	(Подпись)	(Дата)	(И.О. Фамилия)

Архангельск 2026

СОДЕРЖАНИЕ

Перечень сокращений и обозначений	3
Введение	4
1 Анализ и разработка требований	6
1.1 Назначение и область применения	6
1.2 Постановка задачи	6
1.3 Выбор состава программных и технических средств	8
2 Проектирование информационной системы	10
2.1 Проектирование базы данных	10
2.2 Проектирование архитектуры информационной системы	11
3 Разработка информационной системы	13
3.1 Разработка серверной части	13
3.2 Разработка клиентской части	16
3.3 Разграничение прав доступа и обеспечение защиты данных	19
3.4 Тестирование информационной системы	21
4 Инструкция по эксплуатации информационной системы	25
4.1 Установка приложения	25
4.2 Инструкция по работе с информационной системой	25
5 Определение затрат на разработку	34
6 Охрана труда и техника безопасности при работе на компьютере	49
6.1 Общие требования охраны труда	49
6.2 Требования охраны труда перед началом работы	49
6.3 Требования охраны труда во время работы	49
6.4 Требования охраны труда по окончании работы	50
6.5 Требования охраны труда в аварийных ситуациях	50
Заключение	51
Список использованных источников	53

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем дипломном проекте применяются следующие сокращения и обозначения:

БД – база данных

ИС – информационная система

МРОТ – минимальный размер оплаты труда

ПК – персональный компьютер

СУБД – система управления базами данных

ЭВМ – электронная вычислительная машина

CSRF – межсайтовая подделка запроса

CSS – каскадные таблицы стилей

HTML – язык гипертекстовой разметки

HTTP – протокол передачи гипертекста

HTTPS – защищённый протокол передачи гипертекста

JSON – текстовый формат обмена данными

JWT – открытый стандарт для безопасной передачи информации между сторонами в формате JSON

MPA – многостраничное приложение

MVC – модель–представление–контроллер

ORM – объектно-реляционное отображение

SQL – язык структурированных запросов

XSS – межсайтовый скриптинг

ВВЕДЕНИЕ

ИС играют важную роль в современном образовательном процессе, обеспечивая доступ к учебным материалам и автоматизацию проверки знаний. В условиях цифровизации образования особенно востребованы образовательные платформы, которые позволяют не только изучать теоретический материал, но и закреплять знания с помощью интерактивных заданий.

При изучении русского языка важна не только теория, но и регулярная практика, позволяющая постепенно закреплять изученные правила. Традиционные способы обучения не всегда позволяют быстро проверять знания, отслеживать успеваемость и систематизировать результаты обучения. Это создает необходимость разработки ИС, объединяющей учебный материал, тестирование и контроль результатов в одном приложении.

Целью дипломного проектирования является разработка ИС «Грамматиум», предназначенной для изучения правил русского языка, прохождения тестов и контроля успеваемости студентов.

Актуальность дипломного проекта заключается в необходимости систематизации учебных материалов по правилам русского языка, решении проблемы автоматизации процесса предоставления учебной информации и обеспечении пользователей удобными инструментами для изучения и проверки знаний.

Электронные средства обучения позволяют индивидуализировать и дифференцировать процесс обучения, осуществлять контроль с диагностикой ошибок и обратной связью, осуществлять самоконтроль и самокоррекцию учебной деятельности, формировать умение принимать оптимальное решение в различных ситуациях, развивать определенный тип мышления (наглядно-образного, теоретического), усиливать мотивацию обучения, формировать культуру познавательной деятельности. Диалог с компьютером,

при всей его ограниченности и шаблонности, позволяет учащемуся самостоятельно определять темп обучения [18].

Для достижения поставленной цели требуется решить следующие задачи:

- собрать и проанализировать требования к ИС для изучения правил русского языка «Граммариум»;
- определить состав программных и технических средств разрабатываемой ИС;
- разработать архитектуру ИС «Граммариум»;
- выделить объекты и атрибуты предметной области в соответствии с требованиями;
- спроектировать и реализовать БД в соответствии с предметной областью;
- разработать пользовательский интерфейс;
- разработать механизмы взаимодействия клиентской и серверной частей приложения;
- разработать основные программные модули ИС;
- провести функциональное тестирование приложения для проверки его работоспособности и соответствия требованиям;
- проанализировать возможные риски проекта и определить затраты на создание приложения;
- разработать инструкцию по работе с ИС.

1 Анализ и разработка требований

1.1 Назначение и область применения

ИС «Граммариум», разрабатываемая для АКТ (ф) СПбГУТ, предназначена для изучения правил русского языка, организации тестирования и контроля качества усвоения знаний студентами.

Использование системы «Граммариум» позволит упростить доступ к учебным материалам, автоматизировать проверку тестов, сократить время на оценку результатов и повысить качество организации учебного процесса за счёт систематизации учебных материалов в единой информационной среде и выполнения тренировочных тестов с различными типами заданий, направленных на закрепление изученных правил.

1.2 Постановка задачи

В ходе дипломного проектирования необходимо разработать многопользовательскую ИС. Клиентскую часть системы реализовать в виде веб-приложения, предоставляющего доступ к функциональности ИС через удобный пользовательский интерфейс.

Функциональные требования к ИС:

- регистрация и аутентификация пользователей;
- разграничение прав доступа для трёх ролей: студент, преподаватель, администратор;
- предоставление учебных материалов по правилам русского языка в области орфографии, пунктуации и синтаксиса;
- организация тренировочного, контрольного и итогового тестирования студентов;
- автоматическая генерация тестов с учётом уровня сложности (новичок, продвинутый, знаток) и типа задания (выбор одного варианта

ответа из нескольких, выбор нескольких вариантов ответа из списка, свободный ответ в текстовом поле, соотнесение элементов, вставка слова или фразы в текст);

- формирование статистики освоения учебного материала, включающей историю прохождения тестов, прогресс по разделам и рекомендации по повторению правил;

- предоставление преподавателю инструментов для контроля успеваемости студентов, включая просмотр результатов контрольных и итоговых тестов и управление доступом к ним и выгрузку результатов в формате Excel;

- предоставление администратору возможности управления учётными записями пользователей, учебными материалами и тестами.

Эксплуатационные требования к ИС:

- хэширование паролей с использованием алгоритма bcrypt;
- шифрование личных данных пользователей [5];
- разграничение прав доступа на уровне функций и данных;
- защита форм от CSRF и XSS-атак;
- время отклика системы на действия пользователя – не более двух секунд;

- интуитивно понятный интерфейс с минимальной обучаемостью пользователей;

- адаптивный дизайн для компьютеров, планшетов и мобильных устройств;

- поддержка светлой и тёмной темы оформления.

В системе предусмотрены три категории пользователей: студент, преподаватель и администратор. Студент может изучать правила, проходить тренировочные, контрольные и итоговые тесты, просматривать результаты попыток и личную статистику успеваемости. Преподаватель получает доступ к результатам контрольных и итоговых тестов групп, может просматривать ответы студентов, выгружать результаты и управлять доступом групп к

контрольному и итоговому тестированию. Администратор управляет основными данными системы: пользователями, группами, учебными разделами, правилами, тестами и вопросами, а также выполняет импорт и экспорт данных.

1.3 Выбор состава программных и технических средств

В соответствии с целью проектирования необходимо создать многопользовательскую ИС для изучения правил русского языка. Для создания ИС проведён анализ источников [9], [11], [12], [13], [14] и выбраны следующие технологии:

- в качестве СУБД: MySQL 8.0, так как она обеспечивает высокую скорость работы и удобство в управлении данными. Поддерживает современные форматы данных, такие как JSON, и предлагает улучшенные функции безопасности, также она легко интегрируется с другими инструментами и обеспечивает надежную работу;

- для визуального проектирования БД: инструмент MySQL Workbench, поскольку он предоставляет удобные средства для создания структуры БД, разработки ER-диаграмм, настройки связей между таблицами и выполнения SQL-запросов;

- в качестве редактора исходного кода: Visual Studio Code, так как он является удобной средой для разработки и редактирования программного кода. Данный редактор поддерживает работу с различными языками программирования, имеет встроенную подсветку синтаксиса, средства автодополнения, расширения для Python, HTML, CSS и JavaScript, а также инструменты для удобной навигации по проекту;

- для реализации серверной части приложения: язык программирования Python и фреймворк Flask 3.0, поскольку Python обладает простым и понятным синтаксисом, большим количеством библиотек и хорошо подходит для разработки веб-приложений. Flask является лёгким и

гибким веб-фреймворком, позволяющим реализовать маршрутизацию, обработку запросов, взаимодействие с БД и формирование ответов сервера;

- для реализации пользовательского интерфейса: HTML, CSS и JavaScript, а для формирования динамических HTML-страниц на стороне сервера используется шаблонизатор Jinja2. HTML применяется для создания структуры веб-страниц, CSS – для оформления интерфейса, JavaScript – для обеспечения интерактивности элементов системы. Jinja2 позволяет подставлять данные серверной части в HTML-шаблоны и формировать динамическое содержимое пользовательского интерфейса;

- для обеспечения безопасности, изоляции зависимостей, автоматизации развертывания и обеспечения удобства управления серверной частью ИС: система контейнеризации приложений Docker с плагином Docker Compose [19]. Выбранный технологический стек позволит эффективно и безопасно реализовать функциональные возможности ИС и обеспечить высокое качество программного продукта.

Для функционирования системы на стороне клиента достаточны следующие программные и технические средства:

- программное обеспечение: современный веб-браузер, поддерживающий TLS 1.2, HTML5, CSS3 и JavaScript;
- стабильное интернет-соединение со скоростью не менее 1 Мбит/с.

Для развёртывания и функционирования серверной части ИС необходимы следующие технические и программные средства:

- операционная система: дистрибутив Linux с версией ядра выше 6;
- процессор: не менее 2 ядер, 2 ГГц и выше;
- оперативная память: не менее 2 ГБ;
- свободное дисковое пространство: не менее 2 ГБ;
- среда выполнения контейнеров Docker Engine;
- плагин Docker Compose.

2 Проектирование информационной системы

2.1 Проектирование базы данных

При разработке БД для ИС «Граммариум» необходимо спроектировать структуру таким образом, чтобы она обеспечивала хранение учебных материалов, пользовательских данных, тестов, результатов прохождения и информации, необходимой для анализа освоения правил русского языка. БД должна поддерживать рост числа пользователей и объёма учебных материалов, обеспечивать целостность связей между сущностями и минимизировать избыточность данных.

В БД требуется хранить:

- данные о пользователях системы и их ролях;
- сведения об учебных группах студентов;
- разделы русского языка и соответствующие им правила;
- вопросы для тестирования, относящиеся к определённым правилам;
- данные о тестах и их настройках;
- сведения о связи тестов с правилами;
- параметры формирования тестов по типам и сложности вопросов;
- сведения о доступе групп к контрольным и итоговым тестам;
- информацию о попытках прохождения тестов;
- снимки вопросов и ответы пользователей в рамках конкретной попытки;
- данные refresh-токенов, используемых для авторизации и управления пользовательскими сессиями;
- данные, необходимые для расчёта статистики освоения правил и динамики обучения.

БД спроектирована с использованием реляционной модели, что обеспечивает структурированное хранение данных, целостность связей между сущностями и удобную обработку информации [10].

На рисунке 1 представлена физическая модель БД.

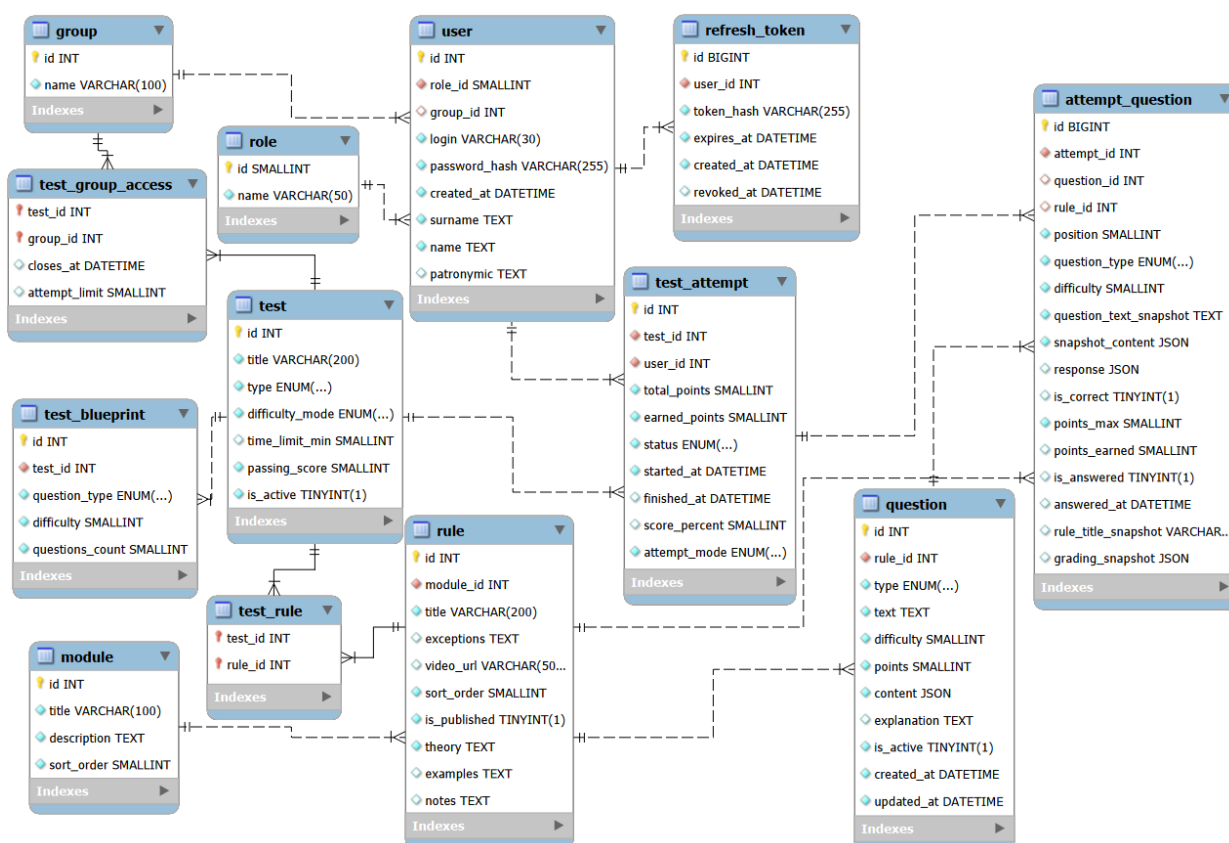


Рисунок 1 – MySQL Workbench. Физическая модель БД

2.2 Проектирование архитектуры информационной системы

Для разработки ИС выбрана трёхуровневая архитектура, включающая клиентский уровень, серверное приложение и уровень БД [8]. Серверная часть реализована как монолитное Flask-приложение, что позволяет централизованно обрабатывать запросы пользователей, обеспечивать доступ к данным и реализовывать бизнес-логику.

Архитектура системы обеспечивает чёткое разделение ответственности между уровнями: клиентский уровень отвечает за пользовательский интерфейс, серверный – за обработку запросов, бизнес-логику и контроль доступа, уровень БД – за централизованное хранение данных и целостность информации в системе.

Серверная часть ИС разработана с использованием принципов архитектурного паттерна MVC [8].

Для разработки клиентской части в виде веб-приложения выбран подход многостраничного приложения МРА [20].

Для обеспечения возможностей фильтрации, шифрования и отслеживания трафика выбран веб-сервер Nginx в качестве обратного прокси-сервера [21].

Диаграмма разворачивания компонентов изображена на рисунке 2.

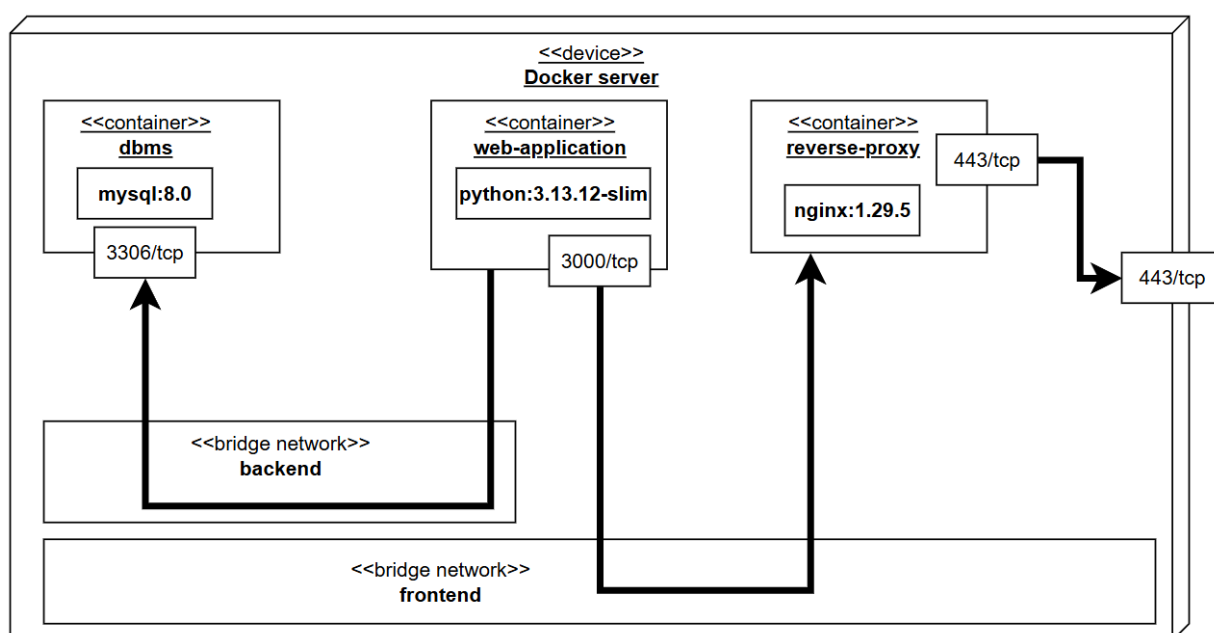


Рисунок 2 – Диаграмма разворачивания компонентов

3 Разработка информационной системы

3.1 Разработка серверной части

Для реализации серверной части приложения выбран фреймворк Flask, который предоставляет инструменты для разработки веб-приложений и отдельных HTTP-эндпоинтов на языке Python [11]. Серверная часть обрабатывает запросы пользователей, выполняет основную логику приложения, взаимодействует с БД и формирует ответы для клиентской части.

Серверная часть приложения построена с использованием MVC-подхода и дополнительного разделения на сервисный и репозиторийный слои. Модели SQLAlchemy описывают структуру данных, Jinja2-шаблоны отвечают за формирование HTML-страниц, а маршруты Flask Blueprint выполняют роль контроллеров. Бизнес-логика вынесена в сервисы, а повторяющиеся запросы к БД – в репозитории.

Для реализации серверной части применены следующие технологии и инструменты:

- Flask Blueprint – механизм разделения маршрутов приложения на независимые модули;
- Flask-Bcrypt – библиотека для безопасного хеширования паролей пользователей;
- SQLAlchemy ORM – инструмент описания моделей данных и выполнения операций с БД через Python-объекты;
- Flask-WTF и WTForms – библиотеки для описания серверных форм, проверки обязательных полей, длины вводимых значений и корректности пользовательского ввода до передачи данных в сервисный слой;
- openpyxl – библиотека для формирования файлов Excel в формате .xlsx;

- Jinja2 – серверный шаблонизатор, используемый для формирования HTML-страниц пользовательского интерфейса;
- PyJWT – библиотека для формирования и проверки JWT-токенов, используемых при авторизации пользователя.

Такая организация делает серверную часть более структурированной и удобной для сопровождения. Маршруты обрабатывают пользовательские запросы, сервисы выполняют бизнес-операции, а репозитории отвечают за доступ к данным. Это позволяет вносить изменения в отдельные компоненты, не затрагивая остальные части системы.

Одним из серверных сценариев является создание попытки прохождения теста. При запуске теста контроллер передаёт управление сервисной функции, которая создаёт запись попытки, подбирает вопросы согласно структуре теста и сохраняет снимок каждого выбранного вопроса. Такой снимок нужен для того, чтобы при обновлении страницы, продолжении попытки или последующем разборе теста пользователь видел тот же состав заданий. Код функции создания попытки представлен в листинге 1.

Листинг 1 – Код функции создания попытки тестирования

```
def start_attempt(user: User, test: Test, attempt_mode: str |  
None = None) -> TestAttempt:  
    # Определяется режим попытки: переданный явно или  
    соответствующий типу теста.  
    mode = attempt_mode or test.type  
    # Создаётся запись попытки для выбранного теста и  
    пользователя.  
    attempt = TestAttempt(  
        test_id=test.id,  
        user_id=user.id,  
        attempt_mode=mode,  
    )  
    db.session.add(attempt)  
    db.session.flush()  
    # Вопросы подбираются по правилам и структуре генерации  
    теста.  
    questions = _pick_questions(test)  
    random.shuffle(questions)  
    total_points = 0
```

```

    for pos, question in enumerate(questions):
        # Для каждого вопроса формируется снимок отображаемого
        # содержимого.
        snapshot = _make_snapshot(question)
        # В состав попытки сохраняется вопрос и его состояние на
        # момент старта.
        attempt_question = AttemptQuestion(
            attempt_id=attempt.id,
            question_id=question.id,
            rule_id=question.rule_id,
            position=pos,
            question_type=question.type,
            difficulty=question.difficulty,
            rule_title_snapshot=question.rule.title if
question.rule else None,
            question_text_snapshot=question.text,
            snapshot_content=snapshot,
            grading_snapshot=dict(question.content or {}),
            points_max=question.points,
        )
        db.session.add(attempt_question)
        total_points += question.points
    # Максимальный балл попытки рассчитывается по выбранным
    # вопросам.
    attempt.total_points = total_points
    db.session.commit()
    return attempt

```

Следующим серверным сценарием является сохранение ответа пользователя. После выбора ответа клиентская часть отправляет данные на сервер, а контроллер передаёт их сервисной функции. Она находит соответствующий вопрос внутри конкретной попытки, проверяет ответ, рассчитывает количество баллов и сохраняет результат. Ответ пользователя хранится непосредственно в записи `AttemptQuestion`. Код функции сохранения ответа представлен в листинге 2.

Листинг 2 – Код функции сохранения ответа

```

def save_answer(attempt: TestAttempt, question_id: int,
response: dict) -> AnswerSaveResult | None:
    # Ответ ищется только среди вопросов конкретной попытки.
    attempt_question = db.session.execute(
        db.select(AttemptQuestion).filter_by(
            attempt_id=attempt.id,
            question_id=question_id,

```

```

    )
    ).scalars().first()
    if not attempt_question:
        logger.warning("AttemptQuestion not found: attempt=%d,
question=%d", attempt.id, question_id)
        return None
    if not attempt_question.grading_snapshot:
        logger.warning("Grading snapshot not found: attempt=%d,
question=%d", attempt.id, question_id)
        return None
    question_type = attempt_question.question_type
    grading = attempt_question.grading_snapshot
    # Пустой ответ не считается выполненным заданием.
    answered = response_has_content(question_type, response)
    is_correct = _check_answer(question_type, grading, response)
    if answered else False
    points_earned = (
        _calc_points(question_type, grading,
attempt_question.points_max, response, is_correct)
        if answered
        else 0
    )
    # Результат проверки сохраняется в записи AttemptQuestion.
    attempt_question.is_answered = answered
    attempt_question.points_earned = points_earned
    attempt_question.response = response if answered else None
    attempt_question.is_correct = is_correct if answered else
None
    attempt_question.answered_at = datetime.now(timezone.utc) if
answered else None
    db.session.commit()
    return AnswerSaveResult(
        is_correct=is_correct,
        points_earned=points_earned,
        points_max=attempt_question.points_max,
        answered=answered,
    )

```

3.2 Разработка клиентской части

Клиентская часть приложения реализована в виде веб-приложения с использованием архитектурного подхода МРА. При таком подходе основные разделы приложения представлены отдельными HTML-страницами, которые формируются на стороне сервера и передаются пользователю в готовом виде.

HTML-страницы пользовательского интерфейса формируются на сервере с использованием шаблонизатора Jinja2. Общая структура

интерфейса задаётся базовым шаблоном, а повторяющиеся элементы вынесены в отдельные шаблоны и макросы. Это позволяет многократно использовать общие компоненты интерфейса и поддерживать единый внешний вид страниц.

Интерактивное поведение отдельных экранов реализуется средствами JavaScript. Скрипты используются для обработки действий пользователя, динамического обновления отдельных элементов страницы, работы с формами, переключения состояний интерфейса и отправки асинхронных запросов без полной перезагрузки страницы.

Для реализации клиентской части применены следующие технологии и библиотеки:

- HTML – язык разметки для формирования структуры страниц;
- CSS – средство описания внешнего вида интерфейса и адаптивной разметки;
- JavaScript – язык программирования для реализации интерактивного поведения страниц;
- htmx – библиотека для частичного обновления содержимого страниц без полной перезагрузки;
- SortableJS – библиотека для реализации drag-and-drop элементов интерфейса;
- Chart.js – библиотека для построения графиков и диаграмм;
- Marked.js – библиотека для преобразования текста, написанного в формате Markdown, в HTML-разметку;
- DOMPurify – библиотека для очистки HTML-разметки перед отображением пользовательского Markdown;
- EasyMDE – редактор Markdown с удобным пользовательским интерфейсом, предпросмотром текста и панелью инструментов форматирования;
- Lucide Icons – набор иконок, используемый в элементах интерфейса.

Выбранный подход МРА позволяет разделить приложение на отдельные страницы, упростить навигацию между разделами и обеспечить стабильную работу интерфейса при взаимодействии пользователя с системой.

Основным элементом взаимодействия пользователя с системой является страница прохождения теста. Она отображает название теста, таймер, индикатор прогресса, навигацию по вопросам и карточки заданий. Для разных типов вопросов используются отдельные блоки интерфейса, подключаемые динамически при формировании страницы в зависимости от типа задания. Такой подход позволяет использовать один общий экран для заданий с выбором ответа, вводом текста, сопоставлением и заполнением пропусков. Фрагмент шаблона страницы прохождения теста представлен в листинге 3.

Листинг 3 – Фрагмент шаблона страницы прохождения теста

```
<!-- Контейнер хранит параметры попытки для JavaScript-модуля
тестирования. -->
<div class="container test-page-container is-booting"
    data-test-take-config
    data-test-id="{{ attempt.test_id }}"
    data-attempt-id="{{ attempt.id }}"
    data-is-training="{{ 'true' if can_check_answers else
'false' }}"
    data-is-preview="{{ 'true' if attempt.attempt_mode ==
'preview' else 'false' }}"
    {% if attempt.attempt_mode == 'preview' %}
    data-preview-discard-url="{{
url_for('dashboard.test_preview_discard', attempt_id=attempt.id)
 }}"
    data-preview-discard-csrf="{{ csrf_token() }}"
    {% endif %}
    data-server-answers='{{ answer_map|tojson|forceescape }}'>
    <div class="test-take-shell">
        <div class="test-take-top">
            <div class="test-page-header">
                <h2 class="test-page-header__title">{{ test.title
 }}</h2>
                {% if deadline_ts %}
                <div class="test-timer" id="timer" data-deadline="{{
deadline_ts }}">
                    {{ icon('clock', 16) }}
```

```

        <span id="timer-display">{{
attempt.test.time_limit_min }}:00</span>
    </div>
    {% endif %}
</div>
<div class="question-nav" id="question-nav">
    {% for aq in questions %}
        <button class="question-nav__dot {% if loop.index == 1
%}active{% endif %}"
            data-index="{{ loop.index0 }}"
            title="Вопрос {{ loop.index }}"></button>
    {% endfor %}
</div>
</div>
<div class="test-question-stage">
    {% for aq in questions %}
        <div class="question-card {% if not loop.first %}hidden{%
endif %}"
            id="question-{{ loop.index0 }}"
            data-index="{{ loop.index0 }}"
            data-aq-id="{{ aq.id }}"
            data-question-id="{{ aq.question_id }}"
            data-type="{{ question_map[aq.question_id].type }}"
            data-answered="{{ 'true' if aq.is_answered else
'false' }}">
            {% set q = question_map[aq.question_id] %}
            <p class="question-card__text">{{ q.text }}</p>
            {% set qtype = q.type %}
            {% set snap = aq.snapshot_content %}
            <!-- Частичный шаблон выбирается по типу вопроса. -->
            {% include 'tests/question_types/' ~ qtype ~ '.html' %}
            {% if can_check_answers %}
            <div class="question-feedback" data-feedback
hidden></div>
            {% endif %}
        </div>
    {% endfor %}
</div>
</div>
</div>

```

3.3 Разграничение прав доступа и обеспечение защиты данных

Разграничение прав доступа в приложении построено на основе JWT-аутентификации и ролей пользователей. После успешной авторизации сервер формирует access token, содержащий идентификатор пользователя, его роль и

срок действия. При последующих запросах сервер проверяет токен и загружает пользователя в контекст текущего запроса.

Для ограничения доступа к отдельным маршрутам используется декоратор `roles_required`. Он принимает список ролей, которым разрешён доступ к маршруту. Если пользователь не авторизован, система перенаправляет его на страницу входа. Если роль пользователя не входит в список разрешённых, сервер запрещает выполнение маршрута и возвращает ошибку доступа. Код декоратора проверки ролей представлен в листинге 4.

Листинг 4 – Код декоратора проверки ролей пользователя

```
def roles_required(*roles):
    # Декоратор принимает список ролей, которым разрешён доступ
    # к маршруту.
    def decorator(f):
        @wraps(f)
        def decorated(*args, **kwargs):
            # Если пользователь не авторизован, возвращается
            # ответ о необходимости входа.
            if not getattr(g, "current_user", None):
                logger.warning("Unauthorized access attempt to
%s", request.path)
                return _unauthorized_response()
            # Если роль пользователя не входит в список
            # разрешённых, доступ запрещается.
            if g.current_user.role.name not in roles:
                logger.warning(
                    "Forbidden access attempt by user %s to %s
(needs: %s)",
                    g.current_user.id,
                    request.path,
                    roles,
                )
                abort(403)
            return f(*args, **kwargs)
        return decorated
    return decorator
```

Помимо `access token` в системе используется `refresh token`. Он применяется для получения новой пары токенов без повторного ввода логина и пароля. В БД сохраняется не исходное значение `refresh token`, а его `bcrypt`-

хеш, что снижает риск использования токена при возможной утечке данных. При выходе пользователя refresh token отзывается, после чего его нельзя использовать для обновления авторизации.

Также одним из методов повышения защищённости, разрабатываемой ИС является изоляция среды выполнения серверного приложения с помощью контейнеризации средствами Docker.

Для обеспечения шифрования передаваемых данных используется веб-сервер Nginx в качестве обратного прокси-сервера с поддержкой HTTPS.

Для защиты персональных данных пользователей применяется симметричное шифрование с использованием механизма Fernet из библиотеки cryptography. Персональные данные сохраняются в БД в зашифрованном виде, а для их шифрования и расшифрования используется ключ DATA_ENCRYPTION_KEY, заданный в файле переменных окружения.

3.4 Тестирование информационной системы

Для оценки качества и надёжности разработанной ИС необходимо провести функциональное тестирование методом «Чёрного ящика» [15]. Тестирование приложения выполняется с целью выявления и устранения потенциальных ошибок, проверки корректности работы всех функциональных возможностей, оценки предсказуемости поведения системы и подтверждения её соответствия требованиям заказчика. В таблице 1 представлен набор тестов для страницы тестов.

Таблица 1 – Набор тестов для страницы тестов

Действие	Ожидаемый результат	Фактический результат
Открыть страницу «Тесты». В фильтре «Раздел» выбрать значение «Орфография»	На экране остаются только карточки тестов, у которых раздел «Орфография». Карточки тестов других разделов скрыты.	Совпадает с ожидаемым результатом

Продолжение таблицы 1

Действие	Ожидаемый результат	Фактический результат
Открыть страницу «Тесты». В фильтре «Тип» выбрать значение «Практика».	На экране остаются только карточки тестов, у которых тип теста «Практика». Карточки тестов других типов скрыты.	Совпадает с ожидаемым результатом
Открыть страницу «Тесты». В фильтре «Сложность» выбрать значение «Продвинутый».	На экране остаются только карточки тестов, у которых тип сложности «Продвинутый». Карточки тестов других сложностей скрыты.	Совпадает с ожидаемым результатом
Открыть страницу «Тесты». В фильтре «Вид» выбрать значение «Практика по правилу»	На экране остаются только карточки тестов, у которых вид теста «Практика по правилу». Карточки тестов других видов скрыты.	Совпадает с ожидаемым результатом
Открыть страницу «Тесты». Последовательно применить фильтры (одновременный выбор фильтра по разделу русского языка, типу теста, виду теста и сложности): «Орфография», «Практика», «Практика по правилу», «Продвинутый».	На экране остаются только карточки тестов, которые одновременно относятся к разделу «Орфография», имеют тип теста «Практика», вид теста «Практика по правилу» и уровень сложности «Продвинутый».	Совпадает с ожидаемым результатом
Открыть страницу «Тесты». Последовательно применить несколько фильтров: «Орфография», «Практика», «Практика по правилу», «Продвинутый». Изменить фильтр «Сложность» с «Продвинутый» на «Новичок».	Список карточек тестов автоматически обновляется с учетом нового значения фильтра, при этом остальные выбранные фильтры продолжают применяться. Фильтр «Сложность» показывает «Новичок».	Совпадает с ожидаемым результатом
Открыть страницу «Тесты». Последовательно применить несколько фильтров: «Орфография», «Практика», «Практика по правилу», «Продвинутый». Сбросить один из выбранных фильтров: выбрать у фильтра «Раздел» значение «Все разделы».	Фильтр «Раздел» снят. Список карточек тестов расширился с учётом остальных фильтров. Фильтр «Раздел» показывает «Все разделы».	Совпадает с ожидаемым результатом

Продолжение таблицы 1

Действие	Ожидаемый результат	Фактический результат
Открыть страницу «Тесты». Последовательно применить несколько фильтров: «Орфография», «Практика», «Практика по правилу», «Продвинутый». Нажать кнопку «Сбросить».	На странице «Тесты» отображаются все доступные карточки тестов.	Совпадает с ожидаемым результатом
Открыть страницу «Тесты». Применить комбинацию фильтров (заведомо не имеющую результата): выбрать у фильтра «Тип» значение «Итоговый», у фильтра «Вид» значение «Практика по правилу».	На странице «Тесты» карточки тестов не отображаются. Отображается сообщение: «Тесты по заданным фильтрам не найдены».	Совпадает с ожидаемым результатом
Открыть страницу «Тесты». Нажать на карточку теста в роли неавторизованного пользователя.	Карточка теста визуально неактивна.	Совпадает с ожидаемым результатом
Стать авторизованным пользователем. Открыть страницу «Тесты». Нажать на карточку теста. Нажать кнопку «Отмена» после появления модального окна.	Модальное окно закрывается. Пользователь остался на странице «Тесты».	Совпадает с ожидаемым результатом
Стать авторизованным пользователем. Открыть страницу «Тесты». Нажать на карточку теста. Нажать кнопку «Начать» после появления модального окна.	Модальное окно закрывается. Пользователь перенаправляется на страницу прохождения теста.	Совпадает с ожидаемым результатом
Стать авторизованным пользователем (студент). Открыть страницу «Тесты». Нажать на карточку открытого контрольного теста.	Появляется модальное окно с названием теста, типом сложности, сколько времени дается на прохождение теста и сообщением: «Вы уверены, что хотите начать тест?», а также кнопки на выбор: «Отмена» и «Начать».	Совпадает с ожидаемым результатом
Стать авторизованным пользователем (студент). Открыть страницу «Тесты». Нажать на карточку закрытого контрольного теста.	Карточка контрольного теста визуально неактивна.	Совпадает с ожидаемым результатом

Продолжение таблицы 1

Действие	Ожидаемый результат	Фактический результат
Стать авторизованным пользователем. Открыть страницу «Тесты». Нажать на карточку незавершенного теста на странице «Тесты».	Появляется модальное окно с названием теста и сообщением: «У вас есть незавершённая попытка. Продолжить выполнение теста?», а также кнопки на выбор: «Отмена», «Начать заново» и «Продолжить».	Совпадает с ожидаемым результатом
Стать авторизованным пользователем. Открыть страницу «Тесты». Нажать на карточку незавершенного теста. Нажать кнопку «Начать заново» после появления модального окна.	Модальное окно закрывается. Пользователь перенаправляется на страницу прохождения теста с новой попытки, а предыдущая попытка автоматически завершается.	Совпадает с ожидаемым результатом
Стать авторизованным пользователем. Открыть страницу «Тесты». Нажать на карточку незавершенного теста. Нажать кнопку «Продолжить» после появления модального окна.	Модальное окно закрывается. Пользователь перенаправляется на страницу прохождения теста предыдущей попытки.	Совпадает с ожидаемым результатом
Стать авторизованным пользователем (преподаватель или администратор). Открыть страницу «Тесты».	На экране отображаются только карточки тестов типа «Практика». Карточки тестов «Контроль» и «Итоговый» скрыты.	Совпадает с ожидаемым результатом

В ходе итогового тестирования не были обнаружены ошибки или отклонения от ожидаемого поведения. Это означает, что текущее состояние страницы с тестами является стабильным и соответствует требованиям.

4 Инструкция по эксплуатации информационной системы

4.1 Установка приложения

Развёртывание ИС заключается в установке и конфигурировании серверной части.

Для развёртывания серверной части ИС на сервере необходимо установить Docker Engine и плагин Docker Compose (версии 5.1.4 или выше). Клонировать репозиторий проекта на сервер и настроить файл «.env». Для запуска серверного приложения необходимо выполнить команду «docker compose up -d» в корне проекта. В результате все необходимые контейнеры будут созданы и запущены.

Конфигурирование серверной части ИС выполняется путём изменения файла переменных среды «.env» в корне проекта. В указанном файле необходимо задать следующие параметры:

- SECRET_KEY – секретный ключ, используемый для защиты служебных данных приложения, flash-сообщений и CSRF-защиты форм;
- DB_CONNECTION_STRING – строка подключения к БД;
- JWT_SECRET_KEY – секретный ключ для формирования и проверки JWT-токенов;
- DATA_ENCRYPTION_KEY – симметричный ключ для шифрования и расшифрования персональных данных пользователей.

4.2 Инструкция по работе с информационной системой

При открытии сайта пользователя встречает главная страница (рисунок 3), на которой представлена информация о назначении сайта, карусель карточек из правил русского языка, поделенная на разделы, а также карточки уровня сложности интерактивных тестов, для быстрого перехода к тестам нужного типа сложности.



Рисунок 3 – Граммариум. Вид главной страницы

Для перехода к правилам требуется нажать кнопку «Правила» в шапке сайта или на главной странице кнопку «Каталог правил». На странице будет представлен список правил русского языка, который можно отфильтровать по разделу. Страница с правилами представлена на рисунке 4.

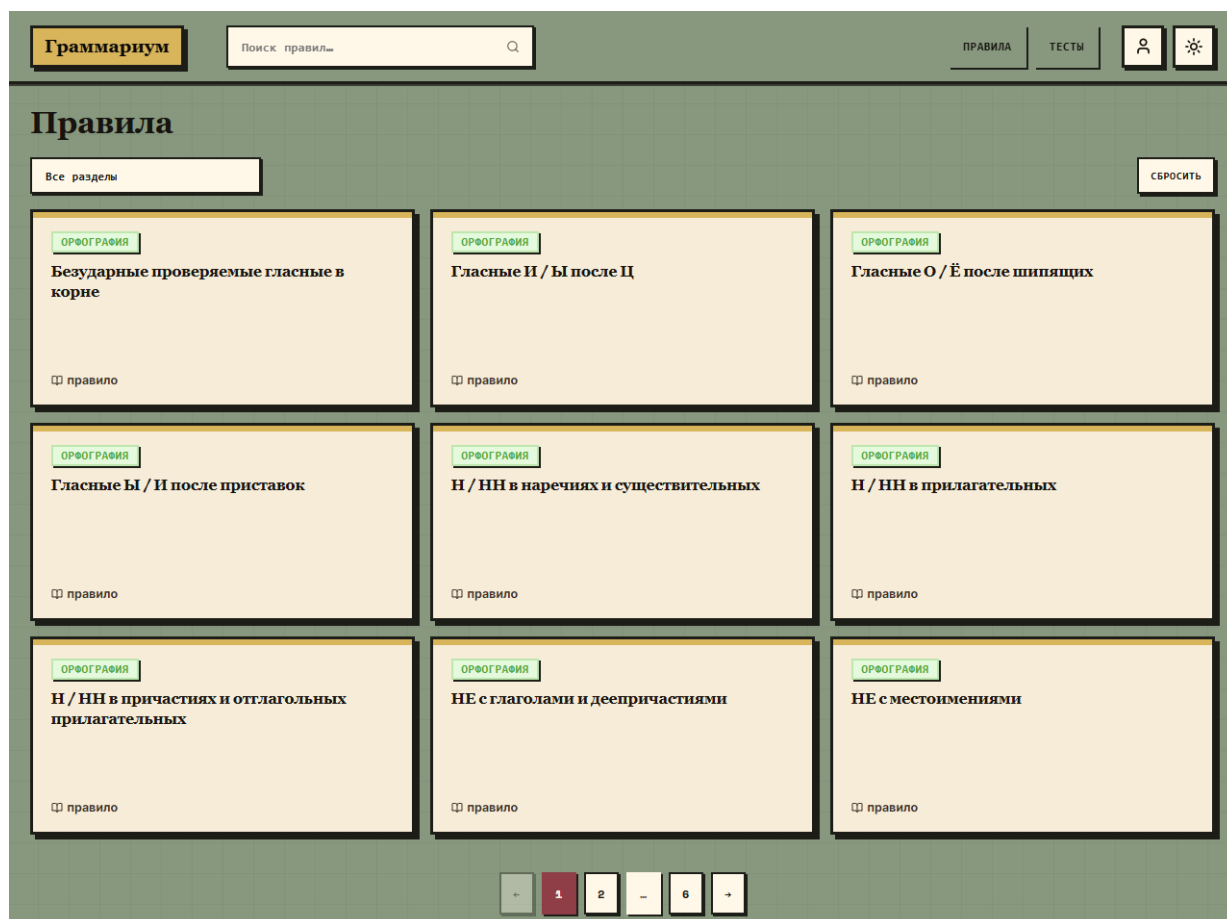


Рисунок 4 – Граммариум. Вид страницы правил

Для изучения правила необходимо нажать на соответствующую карточку. На открывшейся странице представлены теоретический материал и примеры его использования (рисунок 5). Дополнительно могут отображаться исключения, заметки и ссылка на видеоматериал. Также доступны кнопки «Скачать правило» для сохранения материала в PDF-формате и «Пройти тест по правилу» для перехода к тестированию. Навигационные кнопки позволяют перейти к другим правилам.

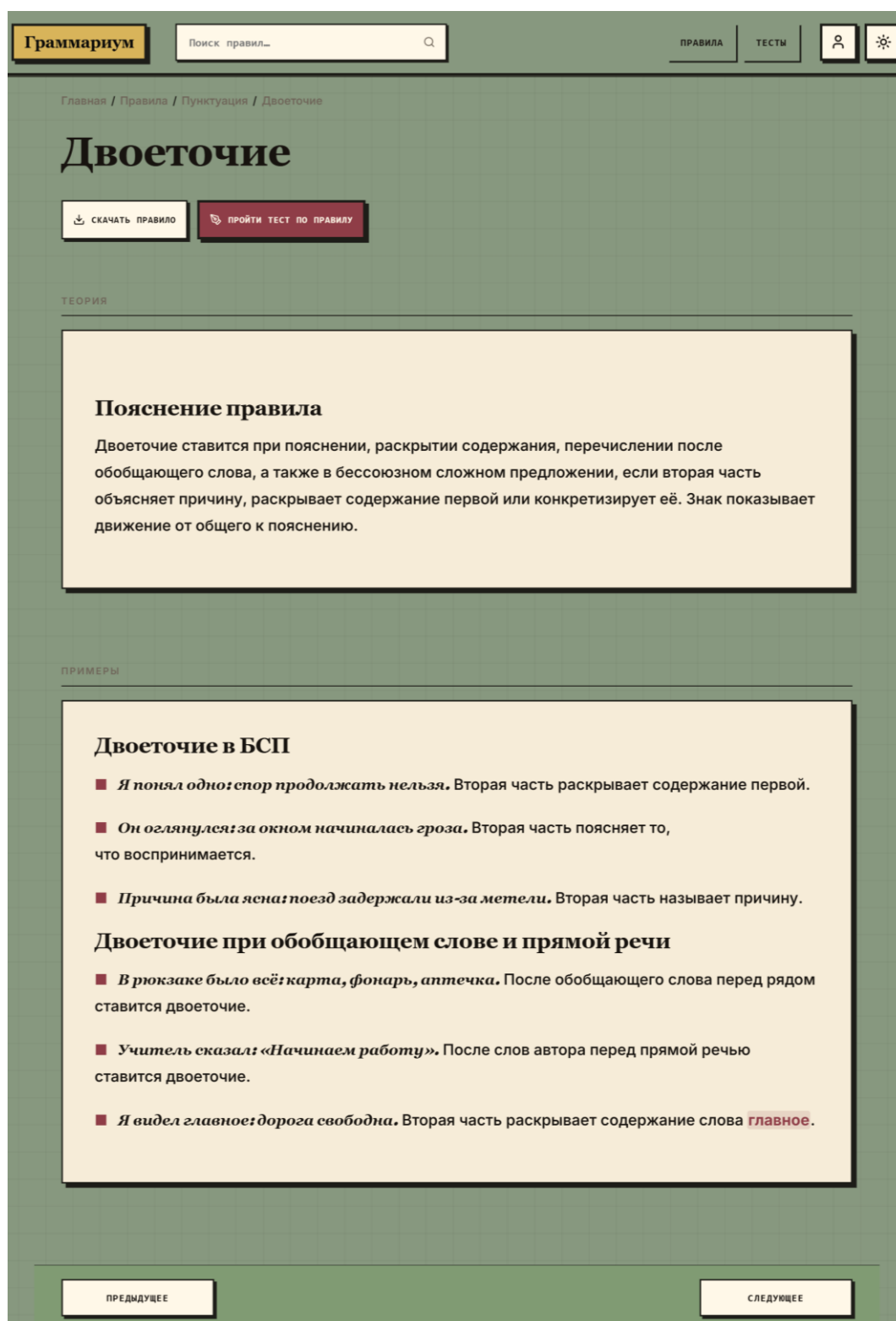


Рисунок 5 – Граммариум. Вид страницы объяснения правил

Переход к списку тестов осуществляется через кнопку «Тесты» в шапке сайта или кнопку «Начать обучение» на главной странице. На странице

тестов представлены карточки заданий и фильтры для их поиска. Страница с тестами показана на рисунке 6.

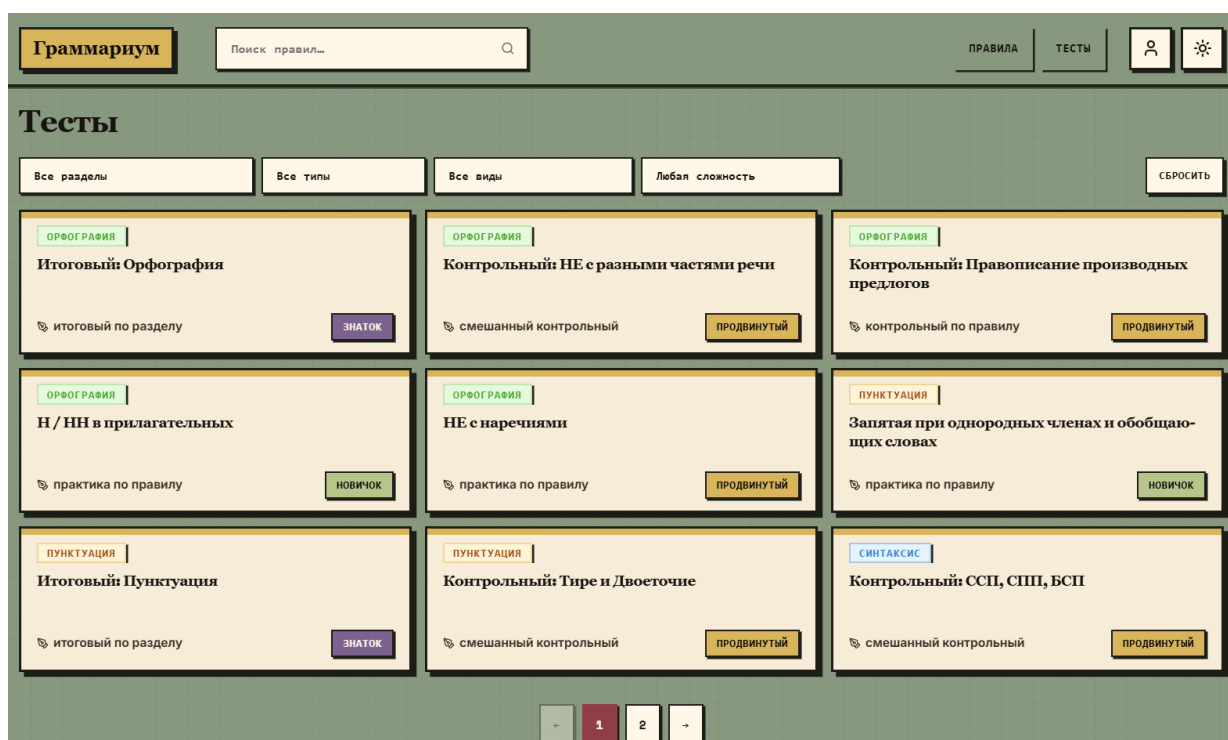


Рисунок 6 – Граммариум. Вид страницы тестов

Для прохождения тестов необходима предварительная авторизация. При выборе теста открывается модальное окно подтверждения, после нажатия кнопки «Начать» осуществляется переход на страницу тестирования (рисунок 7). На ней отображаются название теста, шкала прогресса, элементы быстрого перехода между вопросами, область задания и навигационные кнопки. В тренировочных тестах доступна кнопка «Проверить» для оценки ответа и просмотра ошибки. На последнем вопросе появляется кнопка «Завершить тест», вызывающая подтверждение с указанием количества вопросов без ответа.

После завершения отображаются итоговый результат, список вопросов со статусами ответов, кнопки «Пройти снова» и «К тестам». Для

тренировочных тестов также доступен разбор ответов. Страница результатов теста представлена на рисунке 8.

Граммариум

Поиск правил...

ПРАВИЛА ТЕСТЫ

НЕ с наречиями

Вопрос 3 из 10

Укажите верное написание: «ничуть (не)интересно».

☐ раздельно

☐ слитно

← НАЗАД ПРОВЕРИТЬ ДАЛЕЕ →

Рисунок 7 – Граммариум. Вид страницы прохождения теста

45%

Нужно ещё потренироваться

Баллы: 9 из 20

РЕЗУЛЬТАТЫ ПО ВОПРОСАМ

фрагментах: не с наречиями.

— Распределите слова и формы по орфографическим условиям: не с наречиями.

✓ Выберите правильное написание: «говорить (не)громко».

ПРОЙТИ СНОВА РАЗБОР ТЕСТА К ТЕСТАМ

Рисунок 8 – Граммариум. Вид страницы результатов теста

Для просмотра статистики и прогресса необходимо открыть профиль, нажав на иконку аватара в шапке сайта и выбрав пункт «Профиль». По умолчанию открывается вкладка «Прогресс» (рисунок 9), где отображается статистика обучения за выбранный период, включая количество пройденных тестов, число ответов, среднюю точность, изученные правила и серию дней подряд. Ниже представлены график активности и прогресс по разделам правил, а также блок персональных рекомендаций с рекомендацией дня, правилами для повторения и сильными темами.

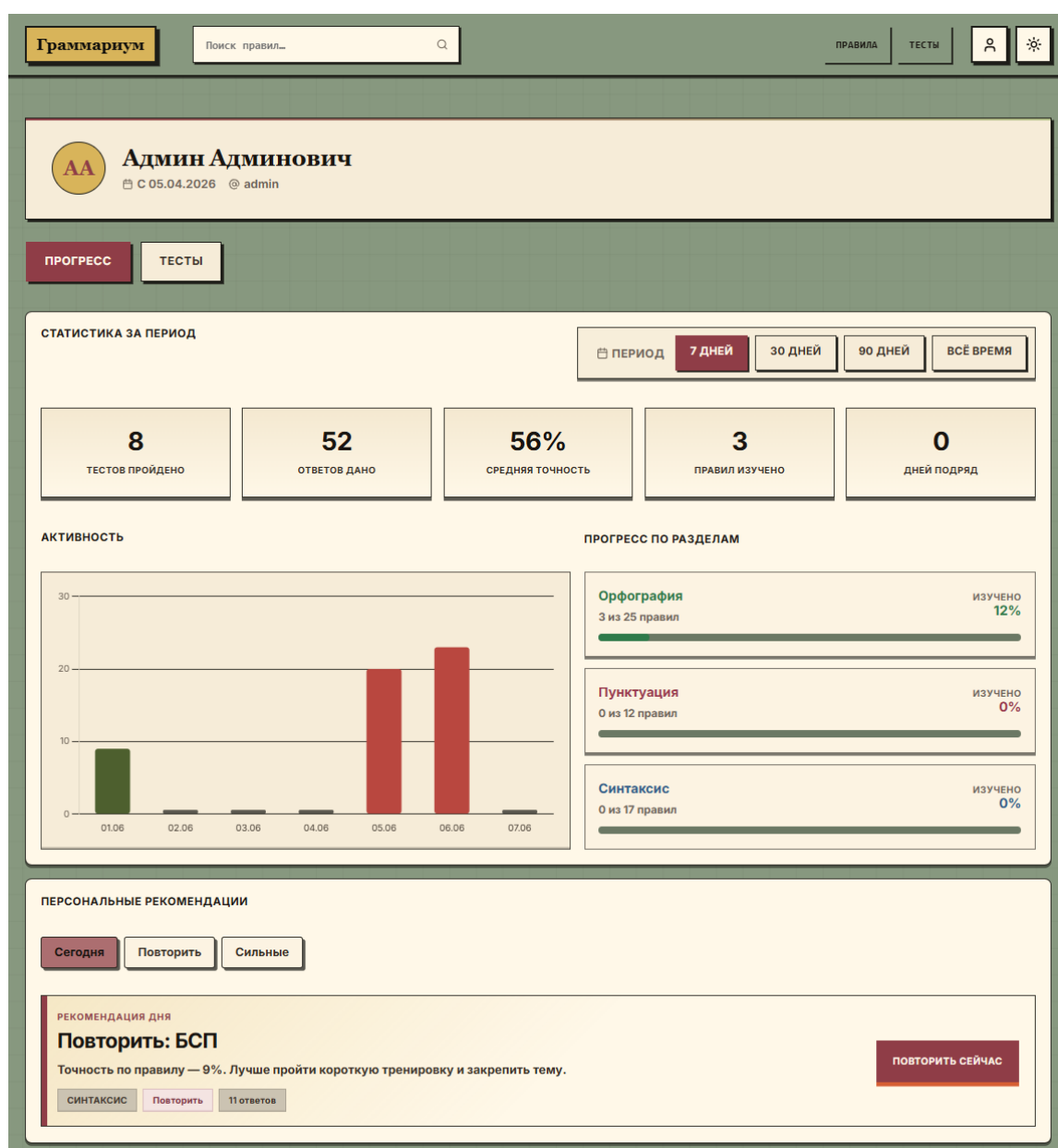


Рисунок 9 – Граммариум. Вид вкладки «Прогресс» в профиле

В разделе «Тесты» (рисунок 10), который находится в профиле, отображается информация о прохождении тестирования. Раздел содержит вкладки «Тренировочные», «Контрольные» и «Итоговые», между которыми можно переключаться для просмотра соответствующего типа тестов. В каждой вкладке при наличии отображаются незавершённые попытки, а также история прохождения тестов выбранного типа. Для тренировочных тестов дополнительно доступен переход к подробному разбору попытки, тогда как для контрольных и итоговых тестов отображаются результаты и список попыток без разбора ответов.

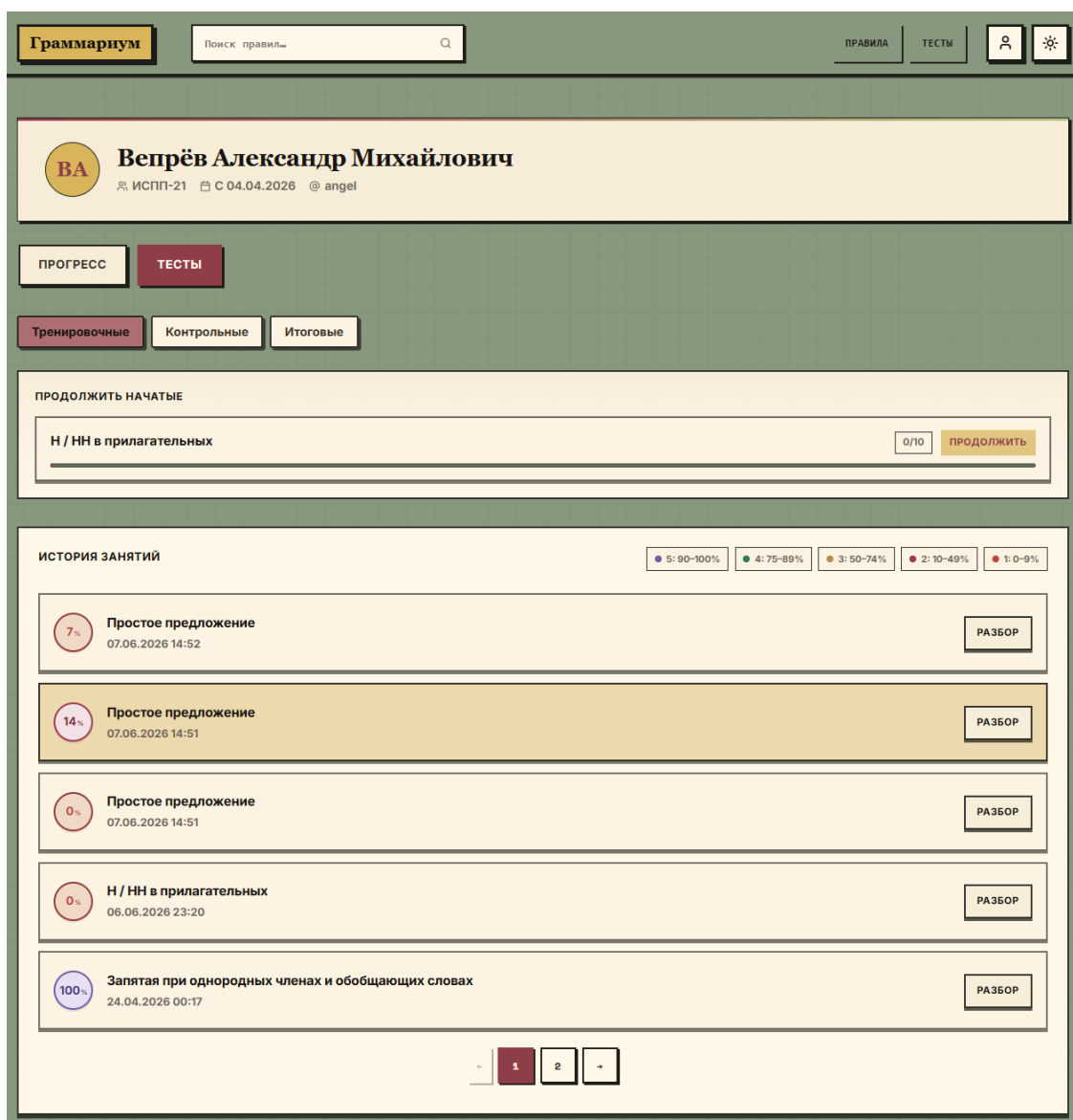


Рисунок 10 – Граммариум. Вид вкладки «Тесты» в профиле

Разбор попытки доступен для тренировочных тестов и предназначен для анализа допущенных ошибок. На странице разбора (рисунок 11) отображается общий результат прохождения, дата попытки, количество вопросов и процент правильных ответов. Ниже приводится список правил, в которых были допущены ошибки, а также подробный перечень вопросов с выбранными и правильными ответами. При наличии пояснения оно отображается под соответствующим вопросом.

[НАЗАД К ИСТОРИИ](#)

Приставки ПРЕ- и ПРИ-

07.06.2026 15:362 ВОПРОСОВ

67%
1 ИЗ 2 ВЕРНО

ОШИБКИ ДОПУЩЕНЫ В ЭТИХ ПРАВИЛАХ

Приставки ПРЕ- и ПРИ-1 ОШИБКА

ВОПРОСЫ

ВОПРОС 1

Отметьте варианты, которые можно считать правильными: приставки пре- и при-.

«пр_дорожная трава» — ПРИ

ВАШ ОТВЕТ - ВЕРНО

«пр_ехать в город» — ПРЕ

«пр_интересный рассказ» — ПРЕ

ВАШ ОТВЕТ - ВЕРНО

«пр_сесть на край» — ПРИ

ВАШ ОТВЕТ - ВЕРНО

«пр_йти домой» — ПРЕ

ВОПРОС 2

Выберите верное написание: «пр_клеить марку».

ПРИ

ПРАВИЛЬНЫЙ

ПРЕ

ВАШ ОТВЕТ

ПОЯСНЕНИЕ

Во фрагменте «пр_клеить марку» нужно выбрать «ПРИ»: ПРИ- употребляется при значении приближения, присоединения, неполноты действия или близости.

Рисунок 11 – Граммариум. Вид страницы разбора попытки

5 Определение затрат на разработку

Затраты на разработку ИС для изучения правил русского языка рассчитаны методом усредненной модели и включают в себя расчет времени (трудозатрат) и денежных средств (расходов) при выполнении данной работы.

Они складываются из средств, затраченных на создание программного продукта, затрат на эксплуатацию ЭВМ и общих затрат, необходимых для написания программы.

Расходы на создание программного продукта рассчитываются по формуле

$$З_{\text{спп}} = З_{\text{спп}}^{\text{МВ}} + З_{\text{общ}}, \quad (1)$$

где $З_{\text{спп}}$ – затраты на создание программного продукта, руб.;

$З_{\text{спп}}^{\text{МВ}}$ – затраты на оплату машинного времени, руб.;

$З_{\text{общ}}$ – общие затраты, руб. ($З_{\text{эвм}}$)

Трудоёмкость разработки программного обеспечения, то есть время на него потраченное, рассчитывается по формуле

$$t = t_o + t_{\text{и}} + t_a + t_{\text{п}} + t_{\text{отл}} + t_{\text{д}}, \quad (2)$$

где t_o - затраты труда на подготовку описания задачи, чел. ч;

$t_{\text{и}}$ - затраты труда на исследование алгоритма решения задачи, чел. ч;

t_a - затраты труда на разработку алгоритма, чел. ч;

$t_{\text{п}}$ - затраты труда на программирование по готовой блок-схеме, чел. ч;

$t_{\text{отл}}$ - затраты труда на отладку программы ЭВМ, чел. ч;

$t_{\text{д}}$ - затраты труда на подготовку документации, чел. ч.

Все составляющие трудозатраты определяются через условное число операторов. Условное число операторов (Q) в программе может быть определено по формуле

$$Q = q \cdot c \cdot (1 + p), \quad (3)$$

где q – число исходных команд (определяется индивидуально для каждого программного продукта);

c – коэффициент, учитывающий новизну и сложность программы;

p – коэффициент коррекции программы в ходе её разработки, зависит от точности и корректности поставленной задачи (от 0,05 до 0,1).

В разработанной программе число операторов составляет примерно 893 ед. ($q = 893$).

Коэффициент, учитывающий новизну и сложность программы (c) определяется исходя из таблицы 2 на пересечении групп сложности и степени новизны.

Таблица 2 – Значение коэффициента c и B

Язык программирования	Группа сложности	Степень новизны				Коэффициент B
		А	Б	В	Г	
Высокого уровня	1	1,38	1,26	1,15	1,20	1,20
	2	1,30	1,19	1,08	0,65	1,35
	3	1,20	1,10	1,00	0,60	1,50
Низкого уровня	1	1,58	1,45	1,32	0,79	1,20
	2	1,49	1,37	1,24	0,74	1,35
	3	1,38	1,26	1,15	0,69	1,50

Программный продукт по степени новизны может быть отнесен к одной из четырех групп:

- группа А – разработка принципиально новых задач;
- группа Б – разработка оригинальных программ;
- группа В – разработка программ с использованием типовых решений;
- группа Г – разовая типовая задача.

По степени сложности программные продукты могут быть отнесены к одной из трех групп:

- алгоритмы оптимизации и моделирования систем;
- задачи учета, отчетности и статистики;
- стандартные алгоритмы.

Программный продукт, разработанный в рамках проекта, можно отнести к группе В по степени новизны, то есть к разработке программ с использованием типовых решений. В то же время по степени сложности алгоритма он относится к задачам учёта, отчётности и статистики. Исходя из этого, согласно таблице 2, коэффициент $c = 1,08$ и коэффициент $B = 1,35$.

Поскольку требования к программному продукту были заранее определены техническим заданием, а изменения в процессе разработки носили уточняющий характер и не требовали существенной переработки структуры приложения, коэффициент коррекции программы в ходе разработки примем равным 0,05 ($p = 0,05$).

Зная исходное число команд (q), для определения условного числа операторов Q воспользуемся формулой (3)

$$Q = 893 \cdot 1,08 \cdot (1,00 + 0,05) = 1012,66 \text{ ед.}$$

Затраты труда на подготовку описания задачи t_0 точно определить невозможно, так как это связано с творческим характером работы. Для примерного расчета примем данное значение равным 45 чел. ч ($t_0 = 45$).

Затраты труда на изучение описания задачи $t_{и}$, чел. ч, с учётом уточнения описания и квалификации программиста можно определить по формуле

$$t_{и} = Q \cdot B / ((75 \dots 85) \cdot K), \quad (4)$$

где Q – условное число операторов;

B – коэффициент увеличения затрат труда вследствие недостаточного или некачественного описания задачи;

K – коэффициент квалификации разработчиков алгоритма и программ.

Согласно таблице 3, для начинающих специалистов, работающих до двух лет, $K = 0,80$

$$t_{и} = 1012,66 \cdot 1,35 / (85 \cdot 0,80) = 20,10 \text{ чел. ч}$$

Т а б л и ц а 3 – Коэффициент квалификации разработчика

Опыт работы	Коэффициент квалификации
До двух лет	0,80
2-3 года	1,00
3-5 лет	1,10 – 1,20
5-7 лет	1,30 – 1,40
Более 7 лет	1,50 – 1,60

Затраты труда на разработку алгоритма решения задачи t_a , чел. ч, определяются по формуле

$$t_a = Q / ((50 \dots 75) \cdot K), \quad (5)$$

$$t_a = 1012,66 / (75 \cdot 0,80) = 16,88 \text{ чел. ч}$$

Затраты времени на составление программы по готовой блок-схеме t_{π} , чел. ч, определяется по формуле

$$t_{\pi} = Q / ((50...75) \cdot K), \quad (6)$$

$$t_{\pi} = 1012,66 / (75 \cdot 0,80) = 16,88 \text{ чел. ч}$$

Затраты труда на отладку программы на ЭВМ при комплексной отладке $t_{отл}$, чел. ч, определяются по формуле

$$t_{отл} = 1,50 \cdot t_{отл}^A, \quad (7)$$

где $t_{отл}^A$ – затраты труда на отладку программы на ЭВМ при автономной отладке одной задачи, чел. ч.

Затраты труда на отладку программы на ЭВМ при автономной отладке одной задачи $t_{отл}^A$, чел. ч, определяется по формуле

$$t_{отл}^A = Q / ((50...75) \cdot K), \quad (8)$$

$$t_{отл}^A = 1012,66 / (75 \cdot 0,8) = 16,88 \text{ чел. ч}$$

Рассчитаем затраты труда на отладку программы на ЭВМ при комплексной отладке по формуле (7)

$$t_{отл} = 1,50 \cdot 16,88 = 25,32 \text{ чел. ч}$$

Расчет затрат труда на подготовку документации – это затраты труда на подготовку и оформление документации по задаче t_d , чел. ч, определяются они по формуле

$$t_d = t_{др} + t_{до}, \quad (9)$$

где $t_{др}$ – затраты труда на подготовку материалов рукописи;

$t_{до}$ – затраты на редактирование, печать и оформление документации.

Затраты труда на подготовку материалов рукописи $t_{др}$, чел. ч, определяются по формуле

$$t_{др} = Q / ((150 \dots 200) \cdot K), \quad (10)$$

$$t_{др} = 1012,66 / (200 \cdot 0,8) = 6,33 \text{ чел. ч}$$

Затраты на редактирование, печать и оформление документации $t_{до}$, чел. ч, определяются по формуле

$$t_{до} = 0,75 \cdot t_{др}, \quad (11)$$

$$t_{до} = 0,75 \cdot 6,33 = 4,75 \text{ чел. ч}$$

Отсюда по формуле (9) находятся затраты труда на подготовку документации

$$t_d = 6,33 + 4,75 = 11,08 \text{ чел. ч}$$

Трудоёмкость разработки программного продукта, согласно формуле (2), составляет

$$t = 45,00 + 20,10 + 16,88 + 16,88 + 25,32 + 11,08 = 135,26 \text{ чел. ч}$$

Итого на разработку всего проекта ушло 135,26 чел. ч, что составляет семнадцать рабочих дней при восьмичасовом рабочем дне.

Таким образом, в результате выполненных расчётов определена общая трудоёмкость проекта, то есть время, затраченное разработчиком на создание программного продукта.

Структура трудозатрат по этапам разработки программного продукта представлена на рисунке 12.

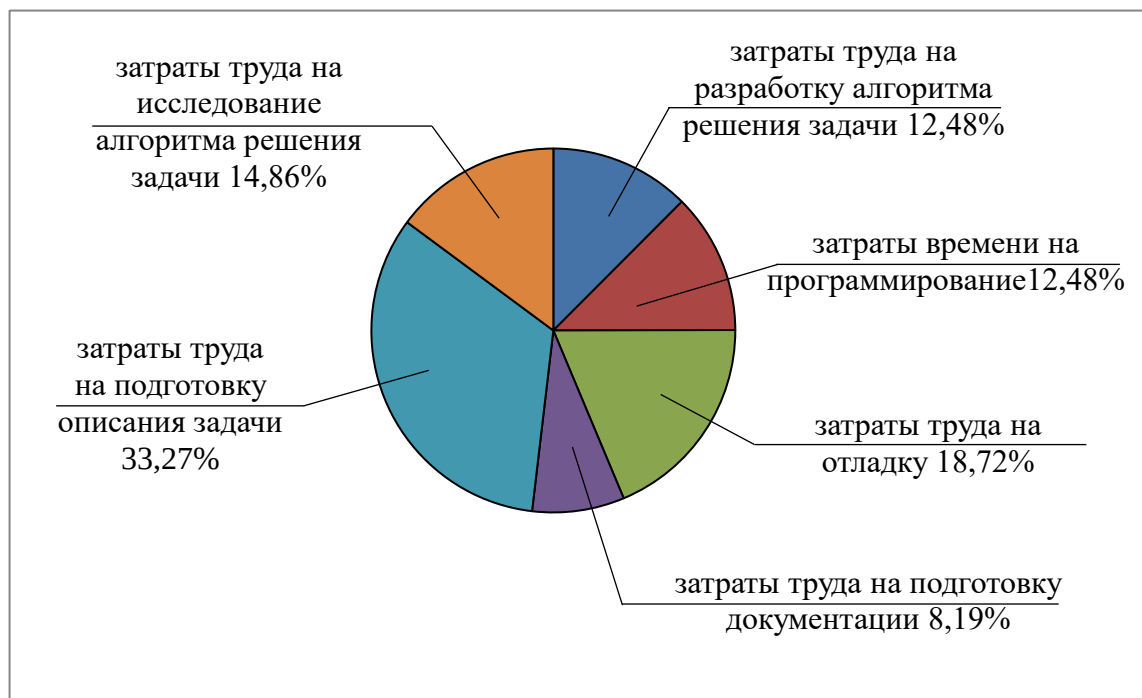


Рисунок 12 – Структура трудозатрат. Диаграмма круговая

Из диаграммы трудозатрат видно, что наибольшая трудоёмкость приходится на подготовку описания задачи – 33,27 % от общей трудоёмкости проекта. Это объясняется необходимостью анализа предметной области, определения требований к программному продукту, исходных данных и ожидаемых результатов.

Затраты на оплату машинного времени при отладке программы $З_{\text{спп}}^{\text{МВ}}$, руб., определяются по формуле

$$З_{\text{спп}}^{\text{МВ}} = C_{\text{час}} \cdot t_{\text{ЭВМ}}, \quad (12)$$

где $C_{\text{час}}$ – цена машино-часа арендного времени, руб/час;

$t_{\text{ЭВМ}}$ – фактическое время отладки программы на ЭВМ, чел. ч.

Фактическое время отладки $t_{\text{ЭВМ}}$, чел. ч, определяется по формуле

$$t_{\text{ЭВМ}} = t_{\text{п}} + t_{\text{отл}} + t_{\text{д}}, \quad (13)$$

$$t_{\text{ЭВМ}} = 16,88 + 25,32 + 11,08 = 53,28 \text{ чел. ч}$$

Цена машино-часа $C_{\text{час}}$, руб/ч, определяется по формуле

$$C_{\text{час}} = З_{\text{ЭВМ}} / T_{\text{ЭВМ}} \quad (14)$$

где $З_{\text{ЭВМ}}$ – полные затраты на эксплуатацию ЭВМ;

$T_{\text{ЭВМ}}$ – действительный месячный фонд времени ЭВМ.

Действительный месячный фонд времени ЭВМ $T_{\text{ЭВМ}}$, ч, определяется по формуле

$$T_{\text{ЭВМ}} = 8 \cdot (K_{\text{д}} - K_{\text{пв}}) - t_{\text{пр}} \cdot 4 \quad (15)$$

где 8 – это ежедневная рабочая норма времени при сорокачасовой рабочей неделе;

$K_{\text{д}}$ – общее количество дней в периоде написания ПП;

$K_{\text{пв}}$ – число праздничных и выходных дней за этот период;

$t_{\text{пр}}$ – время простоя в профилактических работах – это необходимая еженедельная профилактика и цифровая гигиена по четыре часа на протяжении месяца, то есть четырёх недель.

Общее количество календарный дней K_d равно двадцати восьми, число праздничных и выходных дней $K_{\text{пв}}$ равно десяти [17].

Время, затрачиваемое на профилактические работы, составляет четыре часа еженедельно.

Рассчитаем действительный месячный фонд ЭВМ по формуле (15)

$$T_{\text{ЭВМ}} = 8 \cdot (28 - 10) - 4 \cdot 4 = 128 \text{ ч}$$

Общие затраты на эксплуатацию ЭВМ $З_{\text{ЭВМ}}$, руб., определяются по формуле

$$З_{\text{ЭВМ}} = З_{\text{ам}} + З_{\text{эл}} \quad (16)$$

где $З_{\text{ам}}$ – издержки на амортизацию, руб.;

$З_{\text{эл}}$ – издержки на электроэнергию, потребляемую ЭВМ, руб.

Компьютер, используемый для разработки ИС, приобретен за 68 тыс. руб. Амортизируемым имуществом считаются активы, результаты интеллектуальной деятельности и другие объекты интеллектуальной собственности с сроком полезного использования более двенадцати месяцев и первоначальной стоимостью выше 100,00 тыс. руб. [16], следовательно, $З_{\text{ам}} = 0$.

Расчёт затрат на электроэнергию, потребляемую ЭВМ, $З_{\text{эл}}$, руб., определяется по формуле

$$З_{\text{эл}} = P_{\text{ЭВМ}} \cdot T_{\text{ЭВМ}} \cdot C_{\text{эл}}, \quad (17)$$

где $P_{\text{ЭВМ}}$ – суммарная мощность ЭВМ, кВт;

$T_{ЭВМ}$ – время работы компьютера, час;

$C_{эл}$ – стоимость 1 кВт·ч электроэнергии, руб.

Согласно проведённым измерениям, потребляемая мощность компьютера составляет 0,23 кВт. Стоимость 1 кВт·ч электрической энергии, согласно постановлению Агентства по тарифам и ценам Архангельской области от 29 декабря 2025 г. № 75-э/3 «Об установлении цен (тарифов) на электрическую энергию для населения и приравненных к нему категорий потребителей по Архангельской области», составляет 7,54 руб/кВт·ч [7].

Рассчитаем стоимость электроэнергии, потребляемой за месяц по формуле (17) при условии, что разработка программного продукта велась в домашних условиях

$$З_{эл} = 0,23 \cdot 128 \cdot 7,54 = 221,98 \text{ руб.}$$

Тогда общие затраты на эксплуатацию ЭВМ $З_{ЭВМ}$, руб., согласно формуле (16), составят

$$З_{ЭВМ} = 0 + 221,98 = 221,98 \text{ руб.}$$

Рассчитываем цену машино-часа $C_{час}$, руб/ч, по формуле (14)

$$C_{час} = 221,98 / 128 = 1,73 \text{ руб/ч}$$

Расчёт затрат на оплату машинного времени при отладке программы $З_{спп}^{MB}$, руб., определяется по формуле (12)

$$З_{спп}^{MB} = 1,73 \cdot 53,28 = 92,17 \text{ руб.}$$

Расчёт общих затрат на создание программного продукта $З_{общ}$, руб., определяется по формуле

$$З_{\text{общ}} = З_{\text{зп}} + З_{\text{св}} + З_{\text{ЭВМ}} + З_{\text{пр}}, \quad (18)$$

где $З_{\text{зп}}$ – издержки на заработную плату, руб.;

$З_{\text{св}}$ – издержки на страховые взносы с оплаты труда, руб.;

$З_{\text{ЭВМ}}$ – издержки на эксплуатацию ЭВМ, руб.;

$З_{\text{пр}}$ – издержки на прочие и накладные расходы, руб.

Расчёт издержек на заработную плату $З_{\text{зп}}$, руб., определяется по формуле

$$З_{\text{зп}} = \text{оклад} \cdot (1 + \text{РК} + \text{СН}), \quad (19)$$

где РК – районный коэффициент для работающих в местах с определенными климатическими условиями, в нашем случае – районы, приравненные к Крайнему Северу;

СН – коэффициент, учитывающий северную надбавку за стаж работы на территориях, приравненных к Крайнему Северу.

Районный коэффициент составляет 20% от основной заработной платы, а выплаты за выслугу лет, проработанных на территории, приравненной к территории Крайнего Севера – 50% от основной заработной платы [3].

Согласно требованиям трудового кодекса Российской Федерации, месячная заработная плата работника, полностью отработавшего норму рабочего времени и выполнившего трудовые обязанности, не может быть ниже МРОТ [6]. Примем оклад программиста равным величине МРОТ – 27 093,00 руб. [4], [2].

Рассчитываем заработную плату работников по формуле (19)

$$З_{\text{зп}} = 27093,00 \cdot (1 + 0,5 + 0,2) = 46058,10 \text{ руб.}$$

Заработная плата фактическая $З_{\text{ф}}^{\text{зп}}$, руб., определяется по формуле

$$З_{\phi}^{3п} = (З_{3п} / Д_{ср}) \cdot Д_{\phi}, \quad (20)$$

где $Д_{ср}$ – среднее количество рабочих дней в 2026 году;

$Д_{\phi}$ – фактически отработанные рабочие дни.

Рассчитываем фактическую заработную плату $З_{\phi}^{3п}$, руб., по формуле (20)

$$З_{\phi}^{3п} = (46058,10 / 21) \cdot 17 = 37285,13 \text{ руб.}$$

Страховые взносы с оплаты труда $З_{св}$, руб., составляет 30% от суммы зарплаты и определяются по формуле

$$З_{св} = (\Phi_{от} \cdot Н_{св}) / 100, \quad (21)$$

где $\Phi_{от}$ – фонд оплаты труда, руб.;

$Н_{св}$ – актуальный размер страховых взносов, % [1].

Поскольку разработка программного продукта выполняется одним работником – $\Phi_{от}$ принимается равным $З_{\phi}^{3п}$ ($\Phi_{от} = 37285,13$ руб.).

Рассчитываем страховые взносы $З_{св}$, руб., с оплаты труда по формуле (21)

$$З_{св} = (37285,13 \cdot 30) / 100 = 11185,54 \text{ руб.}$$

Прочие затраты $З_{пр}$, руб., принимается в размере 10% от общей суммы затрат и определяется по формуле

$$З_{пр} = ((З_{\phi}^{3п} + З_{св} + З_{ЭВМ}) \cdot 10) / 90, \quad (22)$$

$$З_{пр} = ((37285,13 + 11185,54 + 221,98) \cdot 10) / 90 = 5410,29 \text{ руб.}$$

Рассчитываем общие затраты $Z_{\text{общ}}$, руб., по формуле (18)

$$Z_{\text{общ}} = 37285,13 + 11185,54 + 221,98 + 5410,29 = 54102,94 \text{ руб.}$$

С целью визуализации общих затрат построена диаграмма (рисунок 13).

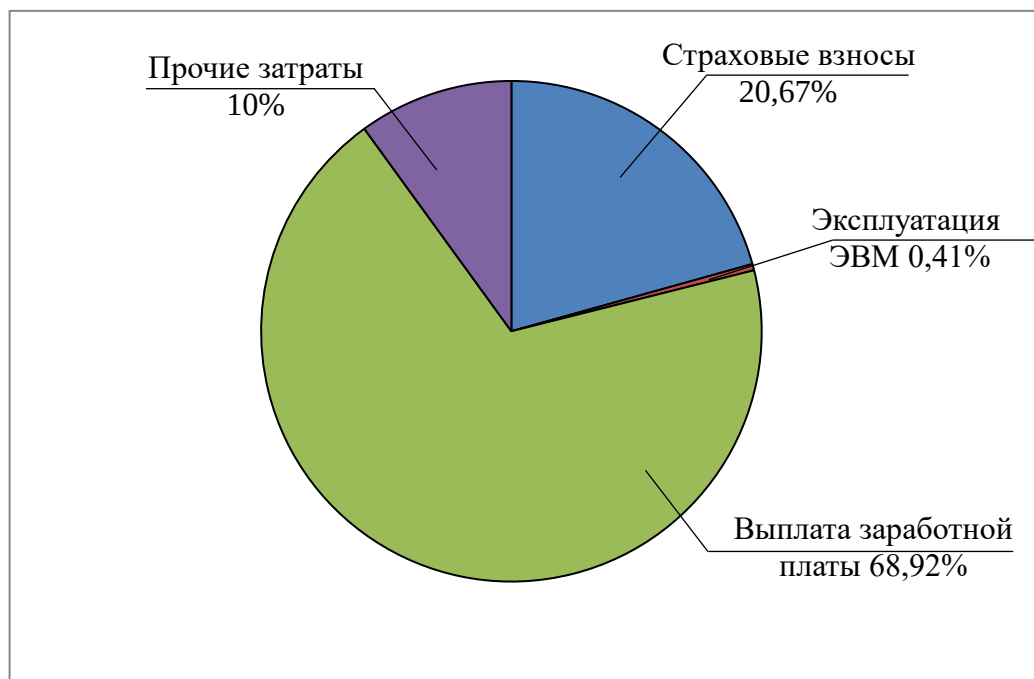


Рисунок 13 – Структура общих затрат. Диаграмма круговая

Рассчитываем затраты на создание программного продукта $Z_{\text{спп}}$, руб., по формуле (1)

$$Z_{\text{спп}} = 92,17 + 54102,94 = 54195,11 \text{ руб.}$$

Во время разработки ИС можно столкнуться с различными рисками, которые могут оказать негативное влияние на разработку, и идентификация которых позволяет разработчику обратить внимание на возможные слабые

места и уязвимости в проекте. Это дает возможность внести необходимые изменения и улучшения, чтобы увеличить качество и надежность проекта.

Наиболее значимые риски определены в таблице 4 и пути их минимизации, не увеличивающие стоимость проекта, установлены.

Т а б л и ц а 4 – Возможные риски и способы их сокращения

Риск	Способ минимизации риска
Технические риски	Проведение своевременного техобслуживания применяемого оборудования
Перебои с электропитанием	Использование имеющегося в наличии источника бесперебойного питания
Риски использования нестабильных технологий	Согласование системных требований с заказчиком
Временная недееспособность разработчика в результате травмы или болезни	Соблюдение правил техники безопасности на рабочем месте и прохождение периодических профилактических медицинских осмотров
Риски плохого взаимодействия между заказчиком и исполнителем	Проведение встреч программистов и заказчиков, уточнения на всех этапах разработки, согласование результатов разработки на каждом этапе
Нарушение сроков разработки продукта	Правильная организация процесса создания программного продукта

Трудозатраты на разработку ИС оценены в 135,26 чел. ч. Согласно выполненным расчётам, затраты на разработку программного продукта составляют 54195,11 руб. Стоимость разработки подобных ИС на рынке в среднем составляет от 70 до 100 тыс. руб. в зависимости от сложности проекта, функциональности и других факторов, влияющих на процесс разработки. Для данного проекта рассчитанные затраты являются конкурентоспособными.

Использование ИС «Граммариум» позволит студентам получать и закреплять знания в области русского языка с помощью теоретических материалов, примеров и интерактивных тестов. Для преподавателей данная система упростит организацию учебного процесса, проведение контроля знаний студентов, просмотр результатов тестирования и оценку успеваемости.

Учитывая рассчитанные трудозатраты, сумму расходов и рыночную стоимость разработки подобных программных продуктов, создание ИС «Граммариум» для АКТ (ф) СПбГУТ является экономически целесообразным. Однако важно тщательно оценить риски, затраты, а также учесть конкурентную среду и потенциальную прибыльность проекта.

Таким образом, разработанная ИС удовлетворяет запросам заказчика и является экономически обоснованной.

6 Охрана труда и техника безопасности при работе на компьютере

6.1 Общие требования охраны труда

Освещённость на рабочем столе должна составлять 300-500 лк. Освещение не должно создавать бликов на экране. В помещении должны использоваться естественные и искусственные источники освещения.

Рабочее место должно быть оборудовано столом шириной 800-1400 мм и глубиной 800-1000 мм, а также подъёмно-поворотным стулом с регулировкой высоты, сиденья и спинки. Расстояние от глаз до экрана должно составлять 600-700 мм, но не менее 500 мм. Клавиатуру следует располагать на расстоянии 100-300 мм от края стола.

На рабочем месте необходимо поддерживать порядок и чистоту, а также проводить систематическое проветривание помещения.

6.2 Требования охраны труда перед началом работы

Перед началом работы с ПК необходимо выполнить следующее:

- подготовить рабочее место;
- отрегулировать освещение на рабочем месте;
- убедиться в отсутствии бликов на экране;
- проверить провода питания и отсутствие оголённых участков;
- отрегулировать положение стола, стула, экрана и клавиатуры в соответствии с требованиями эргономики.

6.3 Требования охраны труда во время работы

Во время работы с ПК запрещается:

- прикасаться к задней панели системного блока при наличии питания;

- переключать разъёмы периферийных устройств при включённом питании;
- допускать попадание влаги на системный блок, монитор, клавиатуру, принтер и другие устройства;
- выполнять самостоятельное вскрытие и ремонт оборудования;
- отключать оборудование от электросети и вынимать штепсельную вилку, держась за шнур;
- работать на компьютере при снятых защитных кожухах.

6.4 Требования охраны труда по окончании работы

При обнаружении неисправности проводов, заземления или запаха гари необходимо немедленно отключить питание и сообщить руководителю.

При возникновении пожара или задымления необходимо:

- обесточить электропитание, закрыть окна и прикрыть двери;
- сообщить в пожарную охрану по телефону 101 или 112, оповестить работающих и руководителя;
- приступить к тушению первичными средствами, если это не сопряжено с риском для жизни;
- покинуть здание и находиться в зоне эвакуации.

При несчастном случае необходимо организовать первую помощь пострадавшему, принять меры по предотвращению развития аварийной ситуации и сохранить обстановку до начала расследования.

6.5 Требования охраны труда в аварийных ситуациях

По окончании работы с ПК требуется:

- отключить ПК от электросети, отключив тумблеры, а также вынуть штепсельную вилку из розетки;
- прибрать рабочее место.

ЗАКЛЮЧЕНИЕ

В последние годы ИС стали неотъемлемой частью образовательного процесса. Разработка ИС охватывает широкий спектр направлений образования, включая школьное обучение, среднее профессиональное и высшее образование. В условиях активной цифровизации учебного процесса, образовательные платформы для интерактивной проверки знаний стали предельно актуальными. Они позволяют не только эффективно усваивать теоретический материал, но и предоставляют удобные инструменты для автоматизированного контроля успеваемости и анализа результатов тестирования.

Целью дипломного проектирования являлась разработка ИС «Грамматиум», предназначенной для изучения правил русского языка, прохождения тестовых заданий и контроля успеваемости студентов.

В ходе работы создана система, объединяющая учебные материалы, тестирование и анализ результатов обучения, обеспечивающая удобный доступ к теоретическому материалу и практической проверке знаний. Разработанное программное обеспечение направлено на повышение эффективности усвоения учебного материала и соответствует требованиям, сформулированным на этапе постановки задачи.

Цель дипломного проектирования достигнута, в процессе её достижения решены следующие задачи:

- собраны и проанализированы требования к ИС «Грамматиум»;
- определён состав программных и технических средств разрабатываемой ИС;
- разработана архитектура ИС «Грамматиум»;
- выделены объекты и атрибуты предметной области в соответствии с требованиями;
- спроектирована и реализована БД в соответствии с предметной областью;

- разработан пользовательский интерфейс;
- разработаны механизмы взаимодействия клиентской и серверной частей приложения;
- разработаны основные программные модули ИС: учебный модуль, модуль тестирования, модуль преподавателя и модуль администрирования;
- проведено функциональное тестирование приложения для проверки его работоспособности и соответствия требованиям;
- проанализированы возможные риски проекта и определены затраты на создание приложения;
- разработана инструкция по работе с ИС.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Российская Федерация. Законы. Налоговый кодекс Российской Федерации. Часть вторая : Федеральный закон № 117-ФЗ : [принят Государственной думой 19 июля 2000 года : одобрен Советом Федерации 26 июля 2000 года]. – Текст : электронный // КонсультантПлюс : [сайт]. – 2026. – URL: https://www.consultant.ru/document/cons_doc_LAW_28165/ (дата обращения: 18.05.2026).

2. Российская Федерация. Законы. О внесении изменения в статью 1 Федерального закона «О минимальном размере оплаты труда» : Федеральный закон № 429-ФЗ : [принят Государственной думой 20 ноября 2025 года : одобрен Советом Федерации 26 ноября 2025 года]. – Текст : электронный // КонсультантПлюс : [сайт]. – 2026. – URL: https://www.consultant.ru/document/cons_doc_LAW_520032/ (дата обращения: 18.05.2026).

3. Российская Федерация. Законы. О государственных гарантиях и компенсациях для лиц, работающих и проживающих в районах Крайнего Севера и приравненных к ним местностях : Федеральный закон № 4520-1 : [принят и одобрен Президентом Российской Федерации Б. Ельциным 19 февраля 1993 года]. – Текст : электронный // КонсультантПлюс : [сайт]. – 2026. – URL: https://www.consultant.ru/document/cons_doc_LAW_1786/ (дата обращения: 18.05.2026).

4. Российская Федерация. Законы. О минимальном размере оплаты труда : Федеральный закон № 82-ФЗ : [принят Государственной Думой 2 июня 2000 года : одобрен Советом Федерации 7 июня 2000 года]. – Текст : электронный // КонсультантПлюс : [сайт]. – 2026. – URL: https://www.consultant.ru/document/cons_doc_LAW_27572/ (дата обращения: 18.05.2026).

5. Российская Федерация. Законы. О персональных данных : Федеральный закон № 152-ФЗ : [принят Государственной Думой 8 июля 2006

года : одобрен Советом Федерации 14 июля 2006 года]. – Текст : электронный // КонсультантПлюс : [сайт]. – 2026. – URL: https://www.consultant.ru/document/cons_doc_LAW_61801/ (дата обращения: 16.05.2026).

6. Российская Федерация. Законы. Трудовой кодекс Российской Федерации : Федеральный закон № 197-ФЗ : [принят Государственной думой 21 декабря 2001 года : одобрен Советом Федерации 26 декабря 2001 года]. – Текст : электронный // КонсультантПлюс : [сайт]. – 2026. – URL: https://www.consultant.ru/document/cons_doc_LAW_34683/ (дата обращения: 18.05.2026).

7. Агентство по тарифам и ценам Архангельской области. Постановление от 29 декабря 2025 г. № 75-э/3 «Об установлении цен (тарифов) на электрическую энергию для населения и приравненных к нему категорий потребителей по Архангельской области» // Официальный интернет-портал правовой информации. – URL: <http://publication.pravo.gov.ru/document/2901202512300013> (дата обращения: 18.05.2026).

8. Гагарина, Л. Г. Проектирование и архитектура программных систем : учебное пособие / Л. Г. Гагарина, А. Р. Федоров, П. А. Федоров. – 2-е изд., испр. и доп. – Москва : ИНФРА-М, 2025. – 334 с. – (Высшее образование). – Текст : электронный. – URL: <https://znanium.ru/catalog/product/1077727> (дата обращения: 19.05.2026). – Режим доступа: по подписке.

9. Гагарина, Л. Г. Технология разработки программного обеспечения : учебное пособие / Л. Г. Гагарина, Е. В. Кокорева, Б. Д. Сидорова-Виснадул ; под ред. Л. Г. Гагариной. – Москва : ФОРУМ : ИНФРА-М, 2023. – 400 с. – Текст : электронный. – URL: <https://znanium.com/catalog/product/1895679> (дата обращения: 19.05.2026). – Режим доступа: по подписке.

10. Голицына, О. Л. Базы данных : учебное пособие / О. Л. Голицына, Н. В. Максимов, И. И. Попов. – 4-е изд., перераб. и доп. – Москва : ФОРУМ : ИНФРА-М, 2023. – 400 с. – (Высшее образование: Бакалавриат). – Текст :

электронный. – URL: <https://znanium.com/catalog/product/1937956> (дата обращения: 18.05.2026). – Режим доступа: по подписке.

11. Гринберг, М. Разработка веб-приложений с использованием Flask на языке Python : практическое руководство / М. Гринберг ; пер. с англ. А. Н. Киселева. – 2-е изд. – Москва : ДМК Пресс, 2023. – 273 с. – Текст : электронный. – URL: <https://znanium.ru/catalog/product/2102598> (дата обращения: 18.05.2026). – Режим доступа: по подписке.

12. Жуков, Р. А. Язык программирования Python. Практикум : учебное пособие / Р. А. Жуков. – Москва : ИНФРА-М, 2023. – 216 с. – Текст : электронный. – URL: <https://znanium.com/catalog/product/1916202> (дата обращения: 19.05.2026). – Режим доступа: по подписке.

13. Мартишин, С. А. Проектирование и реализация баз данных в СУБД MySQL с использованием MySQL Workbench. Методы и средства проектирования информационных систем и технологий. Инструментальные средства информационных систем : учебное пособие / С. А. Мартишин, В. Л. Симонов, М. В. Храпченко. – Москва : ФОРУМ : ИНФРА-М, 2023. – 160 с. – (Среднее профессиональное образование). – Текст : электронный. – URL: <https://znanium.com/catalog/product/1926394> (дата обращения: 19.05.2026). – Режим доступа: по подписке.

14. Никсон, Р. Создаем динамические веб-сайты с помощью PHP, MySQL, JavaScript, CSS и HTML5 / Р. Никсон. – 6-е изд. – Санкт-Петербург : Питер, 2023. – 832 с. – URL: <https://ibooks.ru/bookshelf/386792/reading> (дата обращения: 30.04.2026). – Режим доступа: для зарегистрир. пользователей. – Текст : электронный.

15. Плаксин, М. А. Тестирование и отладка программ для профессионалов будущих и настоящих : учебное пособие / М. А. Плаксин. – 4-е изд. – Москва : Лаборатория знаний, 2020. – 170 с. – Текст : электронный. – URL: <https://znanium.com/catalog/product/1987457> (дата обращения: 21.05.2026). – Режим доступа: по подписке.

16. Правительство Российской Федерации. Постановление от 1 января 2002 г. № 1 «О Классификации основных средств, включаемых в амортизационные группы» [в ред. Постановления от 18 ноября 2022 г.] // КонсультантПлюс. – URL: http://www.consultant.ru/document/cons_doc_LAW_34710/ (дата обращения: 18.05.2026).

17. Производственный календарь на 2026 год. – Текст : электронный // КонсультантПлюс : [сайт]. – 2026. – URL: <https://www.consultant.ru/law/ref/calendar/proizvodstvennye/2026/> (дата обращения: 18.05.2026).

18. Роберт И. В. Современные информационные технологии в образовании: дидактические проблемы; перспективы использования : монография / И. В. Роберт. – 2-е изд., испр. и доп. – Москва : ИИО РАО, 2010

19. Docker overview. – Текст : электронный // Docker Docs : [сайт]. – 2026. – URL: <https://docs.docker.com/get-started/docker-overview/> (дата обращения: 18.05.2026).

20. Multi-Page Application definition. – Текст : электронный // Sanity : [сайт]. – 2026. – URL: <https://www.sanity.io/glossary/multipage-application> (дата обращения: 18.05.2026).

21. NGINX Reverse Proxy. – Текст : электронный // NGINX Documentation : [сайт]. – 2026. – URL: <https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/> (дата обращения: 18.05.2026).